

Server Frameworks

Backend Development

What a Framework Gives You

A backend framework (Node.js/Express, Django, Spring Boot, .NET) provides the routing, request-parsing, and conventions that would otherwise need to be built by hand for every project. Frameworks differ in philosophy — Express is minimal and unopinionated, giving you building blocks to assemble yourself; Django and Spring Boot are more 'batteries included,' with strong conventions for how a project should be organized; .NET sits between the two, opinionated but flexible. None of these differences change the underlying job of a backend — they just change how much structure is handed to you versus how much you build yourself.

Project Structure & Routing

Most frameworks organize a project around the same recurring pieces: entry point (where the server starts and middleware is registered), routes (mapping URLs and HTTP methods to handler functions), controllers/handlers (the actual logic for each route), and models (representing the data the app works with). Routing itself maps a request's method and path to the function that should handle it — e.g. GET /students/:id !' a handler that looks up one student by id. Path parameters (:id) and query strings (?page=2) are the two main ways data arrives via the URL itself, distinct from data sent in the request body.

Middleware, Across Frameworks

Every mainstream backend framework has some version of middleware — code that runs on a request before (or after) the main handler. In Express this is literally called middleware; Django calls it middleware too; Spring Boot uses filters/interceptors; .NET uses its own middleware pipeline. Regardless of naming, the pattern is identical: a chain of functions each request passes through, commonly used for logging, authentication, parsing request bodies, and centralized error handling — understanding this one pattern transfers directly across every framework track in this module.

Choosing a Framework in Practice

In a real job, framework choice is usually already made by the team/company, and the more valuable skill is being able to get productive in whichever framework is already in use — because the underlying concepts (routing, middleware, request/response handling) are the same everywhere. That said, some patterns of choice are common: Node.js/Express for teams wanting a unified JavaScript/TypeScript stack across frontend and backend; Django for teams prioritizing rapid development with strong built-in tooling (admin panel, ORM); Spring Boot for larger enterprise Java environments; .NET for Microsoft-centric environments and enterprise

systems.

Learn More

Express.js Official Documentation

<https://expressjs.com/>

Django Official Documentation

<https://docs.djangoproject.com/>

Spring Boot Official Documentation

<https://spring.io/projects/spring-boot>